torch.nn.utils.prune.global_unstructured

https://pytorch.org/docs/stable/generated/torch.nn.utils.prune.global_unstru

Description:

Globally prunes tensors corresponding to all parameters in parameters by applying the specified pruning_method. Modifies modules in place by: adding a named buffer called name+'_mask' corresponding to the binary mask applied to the parameter name by the pruning method. replacing the parameter name by its pruned version, while the original (unpruned) parameter is stored in a new parameter named name+'_orig'. parameters (Iterable of (module, name) tuples) – parameters of the model to prune in a global fashion, i.e. by aggregating all weights prior to deciding which ones to prune. module must be of type nn.Module, and name must be a string.

Function Signature

```
torch.nn.utils.prune.global_unstructured(parameters,
pruning_method, importance_scores=None, **kwargs)[source]#

Parameters

Raises
```

Code Examples

```
>>> from torch.nn.utils import prune
>>> from collections import OrderedDict
>>> net = nn.Sequential(
...     OrderedDict(
...         [
...             ("first", nn.Linear(10, 4)),
...             ("second", nn.Linear(4, 1)),
...         ]
...     )
... )
>>> parameters_to_prune = (
...     (net.first, "weight"),
...     (net.second, "weight"),
... )
>>> prune.global_unstructured(
...     parameters_to_prune,
...     pruning_method=prune.L1Unstructured,
...     amount=10,
... )
>>> print(sum(torch.nn.utils.parameters_to_vector(net.buffers())
== 0))
tensor(10)
```

Notes & Warnings

NOTE

Note Since global structured pruning doesn't make much sense unless the norm is normalized by the size of the parameter, we now limit the scope of global pruning to unstructured methods.

Detailed Documentation

Documentation

Globally prunes tensors corresponding to all parameters in parameters by applying the specified pruning_method.

Modifies modules in place by:

adding a named buffer called name+'_mask' corresponding to the binary mask applied to the parameter name by the pruning method.

replacing the parameter name by its pruned version, while the original (unpruned) parameter is stored in a new parameter named name+'_orig'.

parameters (Iterable of (module, name) tuples) – parameters of the model to prune in a global fashion, i.e. by aggregating all weights prior to deciding which ones to prune. module must be of type nn.Module, and name must be a string.

pruning_method (function) – a valid pruning function from this module, or a custom one implemented by the user that satisfies the implementation guidelines and has

PRUNING_TYPE='unstructured'.

importance_scores (dict) – a dictionary mapping (module, name) tuples to the corresponding parameter's importance scores tensor. The tensor should be the same shape as the parameter, and is used for computing mask for pruning. If unspecified or None, the parameter will be used in place of its importance scores.

kwargs – other keyword arguments such as: amount (int or float): quantity of parameters to prune across the specified parameters. If float, should be between 0.0 and 1.0 and represent the fraction of parameters to prune. If int, it represents the absolute number of parameters to prune.

TypeError – if PRUNING_TYPE != 'unstructured'

Since global structured pruning doesn't make much sense unless the norm is normalized by the size of the parameter, we now limit the scope of global pruning to unstructured methods.